



Title:-

- Dom XSS in document.write sink using source location.search.
- Dom XSS in innerHTML sink using source Location.search .
- Reflected XSS into attribute with angle brackets HTML-encoded .
- Stored XSS into anchor href attribute with double quotes HTML-encoded .

Prepared By :- Ahmed Dahman Saleh .

Date :- June , 2026 .

Dom XSS in document.write sink using source location.search.

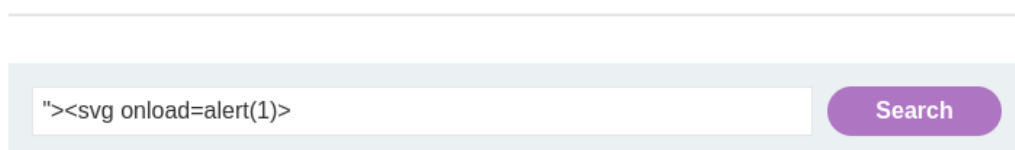
Introduction :-

This report explores a DOM-based Cross-Site Scripting (XSS) vulnerability identified within the application's search query tracking functionality. DOM-based XSS occurs when an application contains client-side JavaScript that processes data from an untrusted source in an unsafe way—usually by writing the data back to the Document Object Model (DOM).

In this specific laboratory exercise, the vulnerability stems from the use of the `document.write` function. This function is a dangerous "sink" because it can interpret its input as HTML, allowing for the injection of arbitrary scripts. The application retrieves user-controlled input directly from the `location.search` property—the "source"—and passes it to `document.write` without proper sanitization or validation. By manipulating the URL parameters, an attacker can influence this data flow to execute arbitrary JavaScript code within the context of the user's browser session.

The objective of this analysis is to demonstrate how this insecure data flow can be leveraged to trigger the `alert()` function, thereby confirming the existence of the vulnerability and highlighting the critical importance of secure input handling and output encoding in client-side development.

Injection of Malicious Payload:-

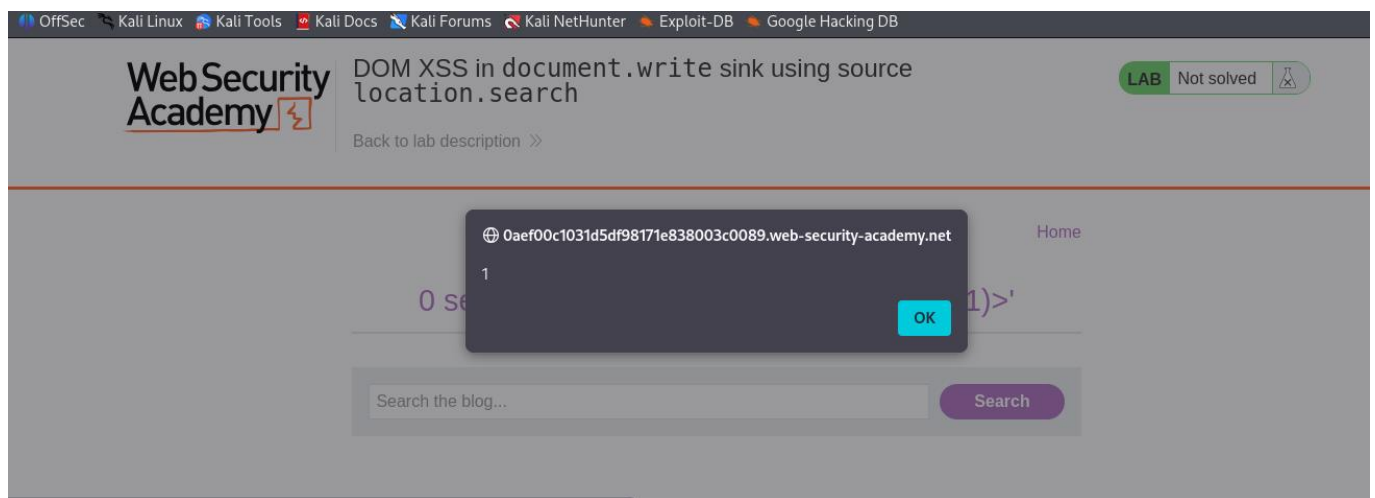


A screenshot of a web application's search interface. It features a light gray rectangular container. Inside, on the left, is a white text input field containing the payload `"><svg onload=alert(1)>`. To the right of the input field is a purple rounded rectangular button with the word "Search" in white text.

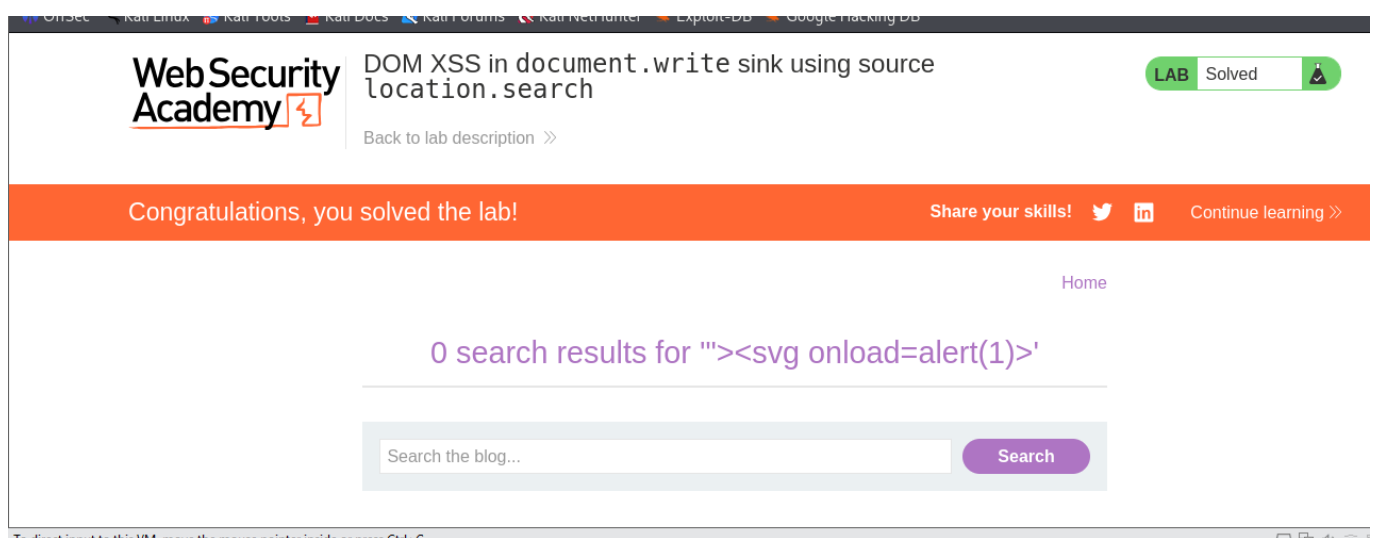
Injecting the XSS payload "><svg onload=alert(1)>" into the application's search input field. This payload is designed to break out of the current HTML context and execute arbitrary JavaScript.

Successful Execution of XSS:-

The browser processes the injected script, confirming the vulnerability by triggering the alert() function. This demonstrates that the application fails to sanitize input sourced from the URL before passing it to the document.write sink.



Laboratory Completion:-



The lab confirms successful exploitation, resulting in the desired outcome of the XSS attack. This highlights the risk of insecure DOM manipulation and the necessity for robust input validation mechanisms.

Introduction: DOM-based XSS in innerHTML Sink:-

This report documents the exploitation of a DOM-based Cross-Site Scripting (DOM XSS) vulnerability found within the application's search functionality. The vulnerability arises from the insecure handling of client-side data, where the application retrieves input from a controllable source—specifically the `location.search` property—and subsequently passes it into an `innerHTML` sink without proper sanitization or validation.

Because `innerHTML` parses the assigned string as HTML, an attacker can inject malicious markup, such as `<img src=x onerror=alert(1)>`. When this input is processed, the browser interprets the injected payload as legitimate DOM elements, leading to arbitrary JavaScript execution in the context of the victim's session. This lab demonstrates the critical importance of avoiding dangerous sinks when processing user-supplied input to prevent client-side code injection attacks.

[Home](#)

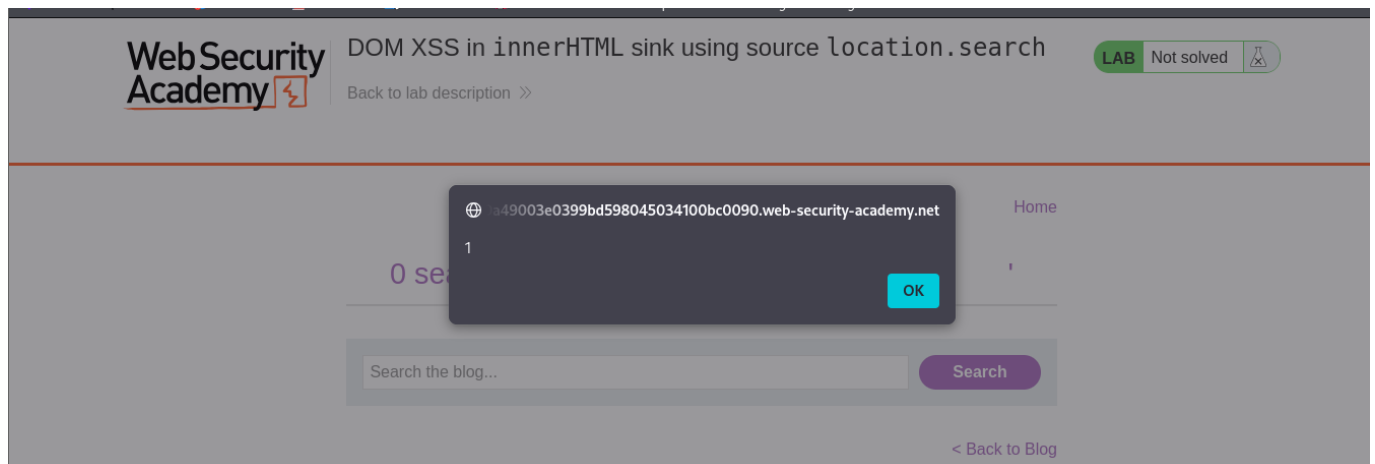
WE LIKE TO
BLOG 



Reconnaissance phase: Inspecting the search field to analyze how user input is processed and reflected by the application.



Injecting a test XSS payload (`<img src=1 onerror=alert(1)>`) into the search parameter to check for DOM-based vulnerabilities.



Successful execution of the payload, confirming that the application is vulnerable to DOM XSS via the `innerHTML` sink.



Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)[Home](#)

Lab successfully solved: Exploitation of the DOM XSS vulnerability confirmed.

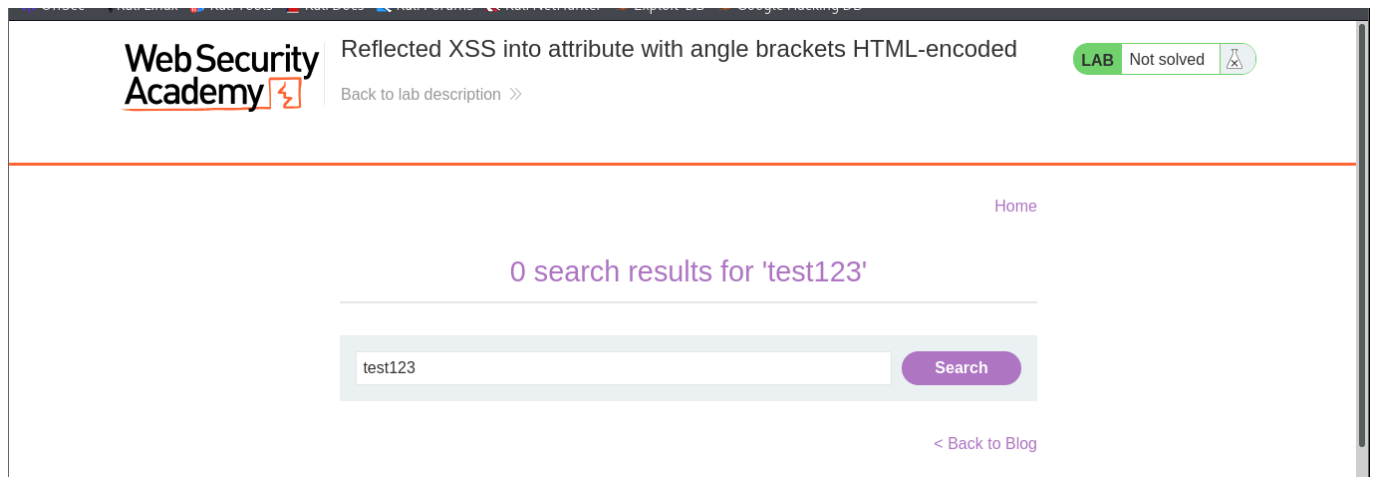
Reflected XSS into attribute with angle brackets HTML-encoded .

Introduction :-

This report details the analysis of a Reflected Cross-Site Scripting (XSS) vulnerability identified within the blog's search functionality. The core challenge of this lab stems from the application's practice of HTML-encoding angle brackets (< and >), which effectively prevents the injection of new HTML tags. However, the user input is reflected directly within an existing HTML attribute, creating a potential for exploitation.

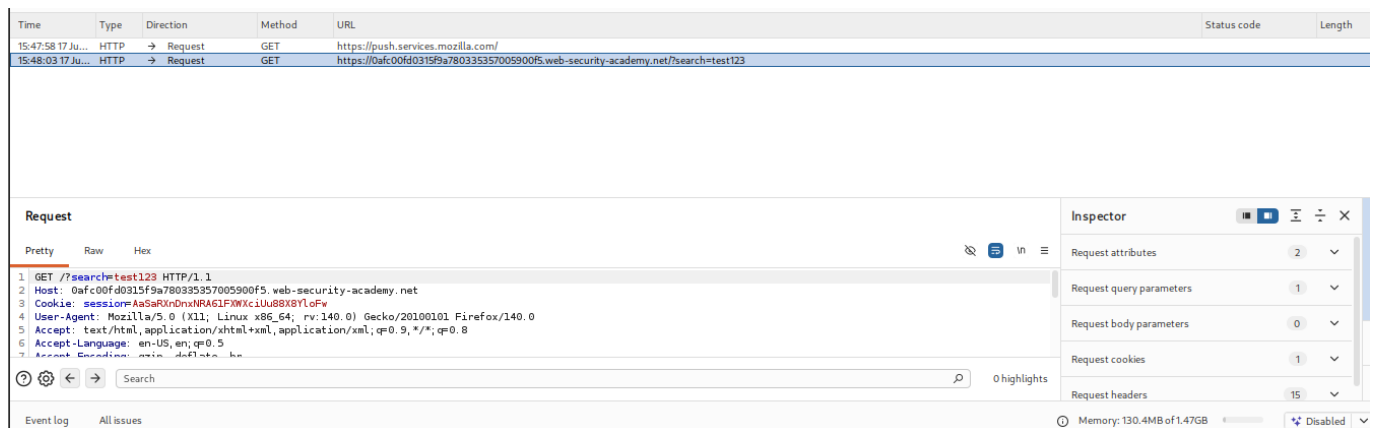
The objective of this lab is to bypass the encoding restrictions by manipulating the execution context of the attribute. This involves injecting a new attribute—specifically an event handler—to execute arbitrary JavaScript, such as the `alert()` function, thereby demonstrating a successful client-side code injection despite the implemented security controls.

Initial Investigation:-



Initial analysis of the target search functionality, where the application reflects the user's input directly into the page.

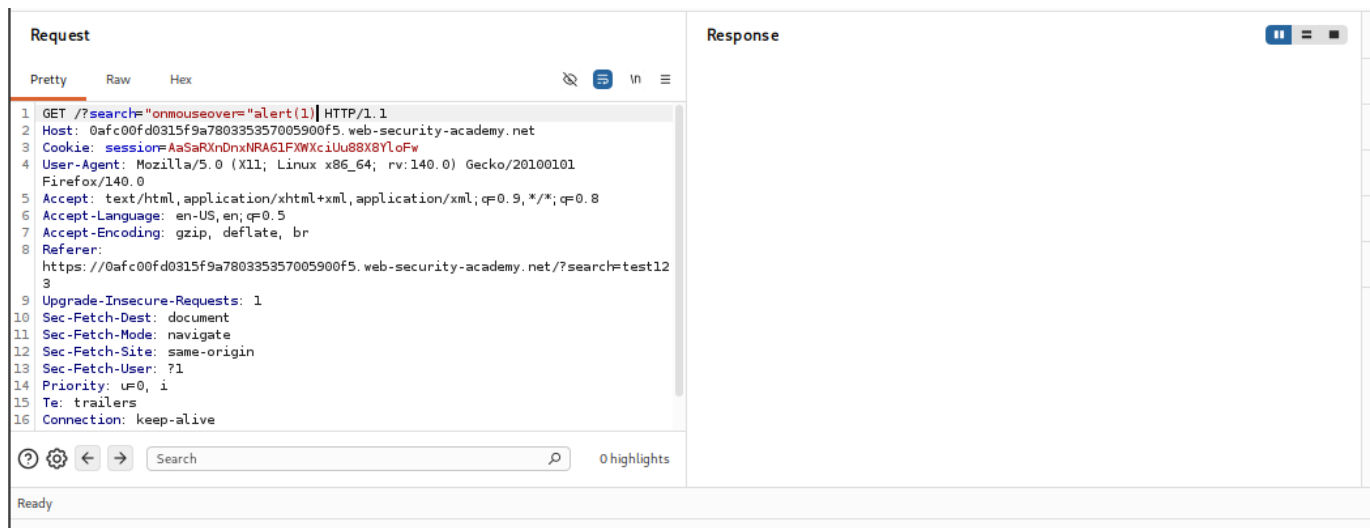
Burp Suite Proxy Capture:-



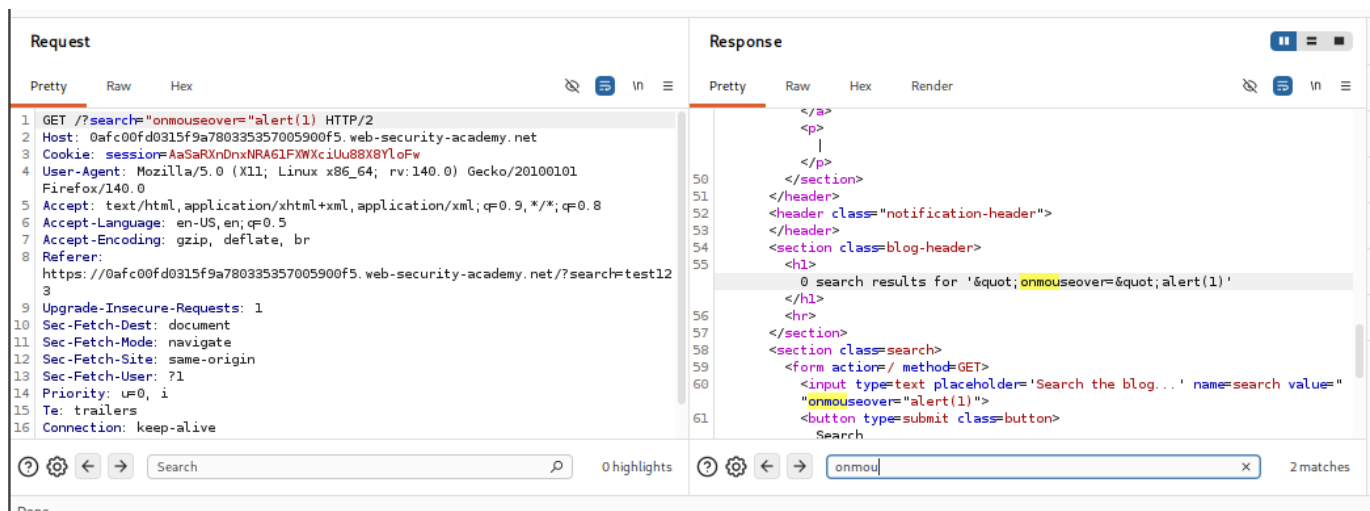
Capturing the HTTP GET request for the search parameter in Burp Suite to analyze how the input is processed and reflected by the server.

Crafting the Malicious Payload:-

Injecting a crafted payload `\\"onmouseover=\\\"alert(1)` to escape the input attribute and introduce an event handler, effectively bypassing the HTML-encoding restriction on angle brackets.



Payload Reflection in HTML Source:-



The HTTP response showing the payload successfully injected into the value attribute of the search `<input>` tag, confirming that the context is correctly broken out.

Successful Exploitation:-

Validation of the XSS vulnerability, confirming the successful execution of the `alert(1)` function, which leads to the completion of the lab.

[OffSec](#) [Kali Linux](#) [Kali Tools](#) [Kali Docs](#) [Kali Forums](#) [Kali NetHunter](#) [Exploit-DB](#) [Google Hacking DB](#)

WebSecurity Academy

Reflected XSS into attribute with angle brackets HTML-encoded

LAB Solved

[Back to lab description >>](#)

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#)

0 search results for 'test123'

[Search](#)

Stored XSS into anchor href attribute with double quotes HTML-encoded .

Introduction:-

Cross-Site Scripting (XSS) vulnerabilities remain one of the most prevalent and critical security risks in modern web applications. The danger of these vulnerabilities lies in their ability to bypass browser security policies, allowing an attacker to execute malicious scripts within the context of a legitimate user's session.

This report provides a detailed analysis of a Stored XSS vulnerability identified within the application's comment functionality. The vulnerability arises from improper sanitization of user-supplied data, specifically within the href attribute of an anchor tag. This flaw allows an attacker to inject arbitrary JavaScript code that executes whenever a victim interacts with the compromised element. Through this analysis, we aim to demonstrate the technical exploitation process, evaluate the potential security impact, and provide the necessary remediation strategies to ensure data integrity and protect end-users.

Initial Interaction with the Vulnerable Input:-

Leave a comment

Comment:

test

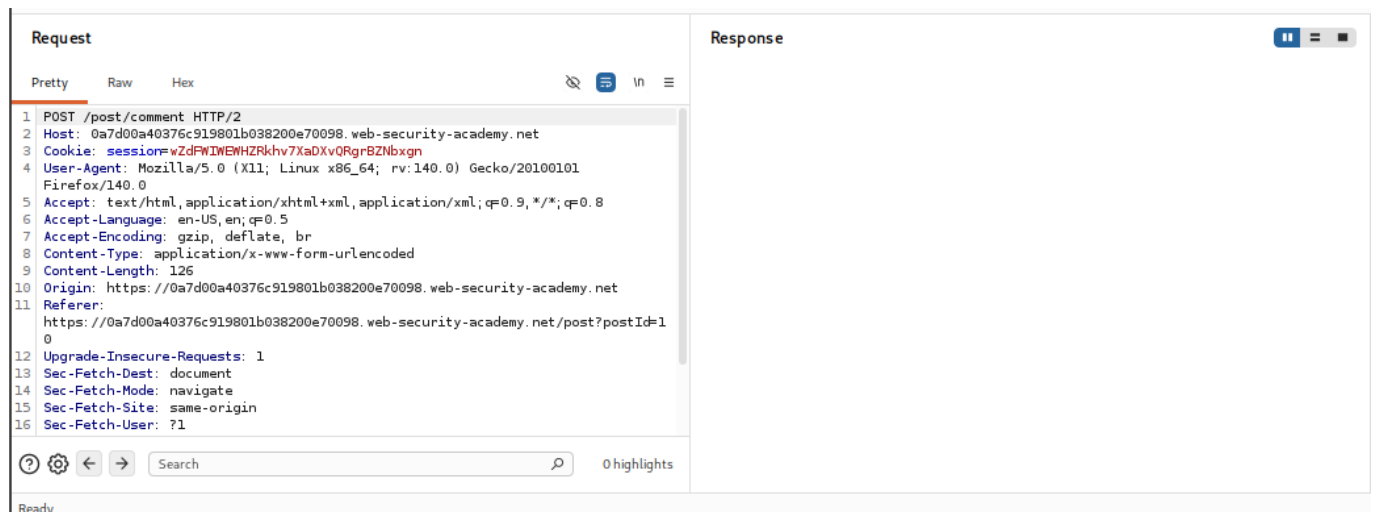
Name:

test

Email:

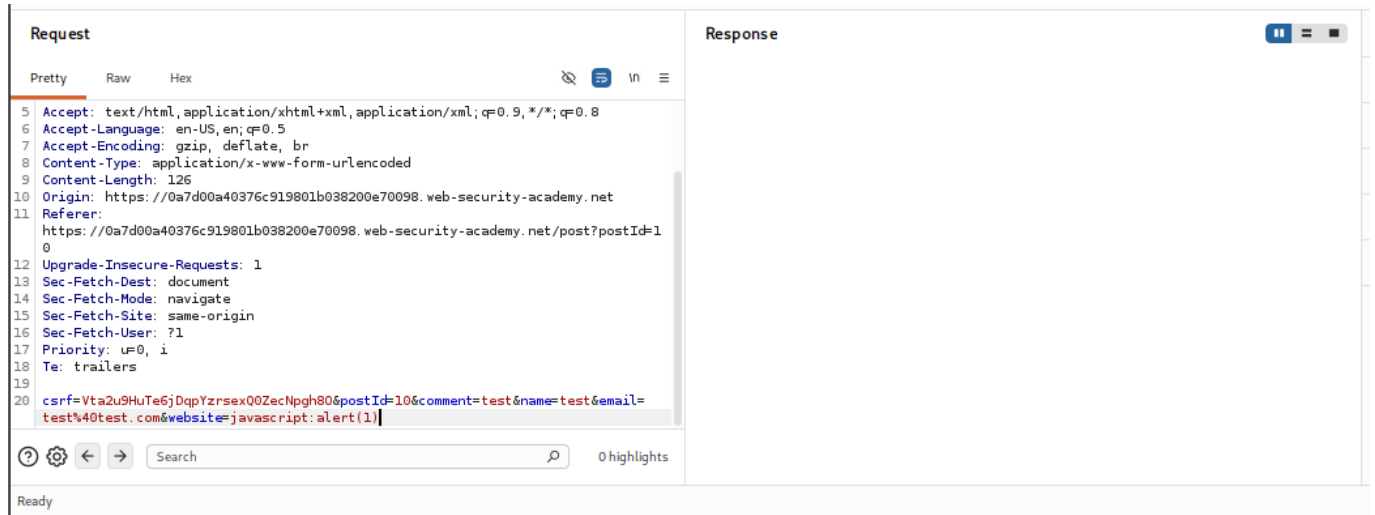
The process begins by identifying the input field susceptible to XSS. In this case, the "Website" field within the comment submission form is targeted as a potential vector for injection.

Capturing the HTTP Request:-



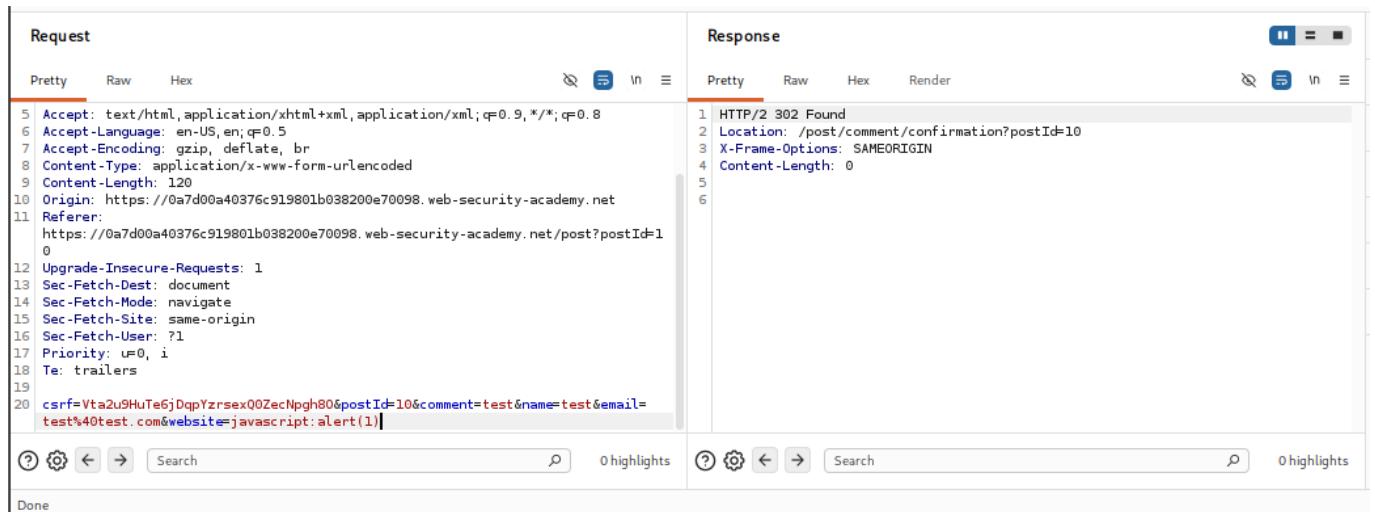
Using an interception proxy (Burp Suite), we capture the POST request sent to `/post/comment`. This allows us to inspect the parameters being submitted to the server and prepare the malicious payload.

Injecting the Malicious Payload:-



The `website` parameter is modified to include the payload `javascript:alert(1)`. This payload is designed to execute JavaScript code when the anchor tag's `href` attribute—which reflects this input—is clicked by a user.

Verification of Server Response:-



The server returns an HTTP/2 302 Found status, indicating that the comment (containing our injected payload) has been successfully submitted and stored on the server side.

Successful Exploitation:-



Stored XSS into anchor href attribute with double quotes HTML-encoded

[Back to lab description](#) >>

LAB Solved 

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning](#) >>

[Home](#)

Thank you for your comment!

Your comment has been submitted.

[< Back to blog](#)

The lab interface confirms that the application has been successfully compromised, indicating the Stored XSS vulnerability was triggered. The payload is now persisted, and the `alert(1)` function will execute whenever the comment author's link is clicked.
